CYLIXLAB · ASSESSMENT

Module 02

Deep Learning & Neural Network Architectures

MCQ Assessment | 30 technical questions with green-check marked answers and full explanations

Assessment overview

Part	Questions	Topic
A	Q1–Q5	MLP: non-linearity, parameter counts, shapes, universal approximation, initialization
B	Q6–Q10	Backpropagation: chain rule, softmax-CE gradient, vanishing / exploding gradients
C	Q11–Q15	Activation functions: ReLU, sigmoid, dying ReLU, multi-label outputs, softmax
D	Q16–Q20	CNNs: output size, parameter counts, pooling, weight sharing, ResNet
E	Q21–Q25	RNNs: BPTT, LSTM gates and parameters, task types, bidirectional models
F	Q26–Q30	Loss functions: cross-entropy, Huber, pairing with outputs, logits pitfalls

HOW TO USE THIS ASSESSMENT

1. Cover the answers and choose one option per question (about 90 seconds each).
2. Check your answer; the correct option is highlighted and marked with a green check.
3. Read **every** explanation, including for questions you got right: the “why the others fail” notes target common misconceptions.
4. Score yourself with the answer key on the last page and revisit the matching theory section.

A Multi-Layer Perceptron

Q1

MLP

A 5-layer MLP is built from dense layers that all use the **identity (linear) activation**. What is the consequence for the model?

- **A.** It becomes slower to train but can still represent non-linear functions
- **B.** All gradients become exactly zero
- ✓ **C. The whole network is equivalent to a single linear transformation, so depth adds no representational power**
- **D.** Its outputs are restricted to the range [0, 1]

Correct answer: C

Why it is correct. A composition of linear maps is linear: $W_5(\dots(W_2(W_1x))) = (W_5\dots W_2W_1)x$. Five layers therefore collapse into **one** matrix and the model is just linear regression (or logistic regression with a sigmoid output). Non-linear activations are what make depth useful.

Why the others fail.

✗ **A.** Training speed is unaffected in principle; more importantly, the function class is still only linear, however slowly it trains.

✗ **B.** Gradients of a linear network are generally non-zero; they just cannot lead to a non-linear model.

✗ **D.** Identity activations leave values unbounded; [0, 1] is the range of a sigmoid.

Q2

MLP

How many **trainable parameters** (weights and biases) does a dense MLP with layer sizes **100 → 50 → 10** have?

- ✓ **A. 5,560**
- **B.** 5,500
- **C.** 5,050
- **D.** 5,610

Correct answer: A

Why it is correct. Layer 1: 100·50 weights + 50 biases = 5,050. Layer 2: 50·10 weights + 10 biases = 510. Total = 5,050 + 510 = **5,560**. General rule: $n_{in} \cdot n_{out} + n_{out}$ per dense layer.

Why the others fail.

✗ **B.** 5,500 counts the weights only (5,000 + 500) and forgets every bias.

✗ **C.** 5,050 is the first layer alone.

✗ **D.** 5,610 over-counts by adding the 50 hidden biases a second time.

Q3

MLP

A dense layer receives a vector of **4 inputs** and has **3 neurons**. Using the convention z = W a + b with column vectors, what is the shape of W?

- **A.** 4 × 3
- **B.** 3 × 3
- **C.** 12 × 1
- **D. 3 × 4** ✓

Correct answer: D

Why it is correct. Each of the 3 neurons has one weight per input, so W has one **row per neuron** and one column per input: shape $(n_{out} \times n_{in}) = 3 \times 4$. Then W a is $(3 \times 4)(4 \times 1) = 3 \times 1$, matching b.

Why the others fail.

✗ **A.** 4 × 3 would be correct only for the row-vector convention z = a W (used in some libraries), not the stated one.

✗ **B.** A 3 × 3 matrix cannot multiply a 4-element input vector.

✗ **C.** A 12 × 1 vector holds the right number of weights but cannot be multiplied with a to give 3 outputs.

Q4

MLP

Which statement correctly describes the **Universal Approximation Theorem**?

- **A.** Any MLP can be trained to zero error with gradient descent
- **B. A one-hidden-layer network with enough neurons and a non-linear activation can approximate any continuous function on a compact domain, but training is not guaranteed to find the weights** ✓
- **C.** Deep networks are never necessary because one layer is always sufficient in practice
- **D.** It requires linear activations to hold

Correct answer: B

Why it is correct. The theorem is an **existence** result: suitable weights exist. It says nothing about how many neurons are needed (may be astronomically many), whether gradient descent will find them, or how well the model generalizes. In practice, depth often gives far more efficient representations.

Why the others fail.

✗ **A.** No optimizer guarantees zero error: loss surfaces are non-convex and data may be noisy or contradictory.

✗ **C.** The theorem does not say a single layer is *practical*; width can explode compared with a deeper network.

✗ **D.** A non-linear activation is essential; with linear activations only linear functions are possible.

Q5 MLP

All weights of a hidden layer are initialized to the **same constant** (for example 0.5). What is the main problem?

- **A.** The gradients explode in the first step
- **B.** Only the bias terms can be learned
- **C.** The loss becomes negative
- ✓ **D. Symmetry: every neuron computes the same output and receives the same gradient, so they never become different**

Correct answer: D

Why it is correct. Identical weights give identical activations, identical backpropagated errors and therefore identical updates, forever. The layer behaves like **one** neuron. Random initialization (He for ReLU, Xavier for tanh/sigmoid) **breaks the symmetry**.

Why the others fail.

✗ **A.** Constant weights do not by themselves cause explosion; the issue is lack of diversity, not scale.
✗ **B.** Weights are updated like any parameter; the problem is that all updates are identical.
✗ **C.** Standard losses such as cross-entropy and MSE are never negative, regardless of initialization.

B Backpropagation

Q6 BACKPROPAGATION

What does **backpropagation** actually compute?

- **A.** The optimal weights directly, in one step
- ✓ **B. The gradient of the loss with respect to every parameter, efficiently, by applying the chain rule backwards and reusing intermediate values**
- **C.** The best learning rate for each layer
- **D.** The global minimum of the loss

Correct answer: B

Why it is correct. Backprop is an algorithm for **computing gradients** at roughly the cost of one extra forward pass. A separate **optimizer** (SGD, Adam) then uses those gradients to update the weights.

Why the others fail.

✗ **A.** Gradients only give a local direction; many update steps are needed.
✗ **C.** Learning rates are hyper-parameters chosen by the practitioner or a schedule, not computed by backprop.
✗ **D.** For non-convex losses neither backprop nor gradient descent guarantees the global minimum.

Q7 BACKPROPAGATION

A single neuron computes a = w·x with **w = 2, x = 3**, and the loss is **L = (a − y)²** with target **y = 4**. What is ∂L/∂w?

- **A.** 4
- **B.** 6
- ✓ **C. 12**
- **D.** 24

Correct answer: C

Why it is correct. Forward: a = 2·3 = 6. Chain rule: ∂L/∂w = (∂L/∂a)(∂a/∂w) = 2(a − y)·x = 2(6 − 4)·3 = 4·3 = **12**. The positive gradient means increasing w would increase the loss, so gradient descent will decrease w.

Why the others fail.

✗ **A.** 4 is only ∂L/∂a; it is missing the factor ∂a/∂w = x.
✗ **B.** 6 is the neuron's output a, not a gradient.
✗ **D.** 24 doubles the correct result, e.g. by applying the factor 2 from the square twice.

Q8 BACKPROPAGATION

A 3-class network uses softmax and cross-entropy. The predicted probabilities are **[0.7, 0.2, 0.1]** and the true class is the **second** one (one-hot [0, 1, 0]). What is ∂L/∂z, the gradient with respect to the logits?

- ✓ **A. [0.7, −0.8, 0.1]**
- **B.** [0.7, 0.8, 0.1]
- **C.** [−0.7, 0.8, −0.1]
- **D.** [0, 1, 0]

Correct answer: A

Why it is correct. For softmax + cross-entropy the gradient simplifies to **ŷ − y** = [0.7 − 0, 0.2 − 1, 0.1 − 0] = [0.7, −0.8, 0.1]. A gradient-descent step lowers the wrong-class logits (positive entries) and raises the true-class logit (negative entry).

Why the others fail.

✗ **B.** The true-class entry must be 0.2 − 1 = −0.8; using +0.8 would push the true class's logit down.
✗ **C.** This is y − ŷ, the negative of the gradient; following it with gradient descent would increase the loss.
✗ **D.** [0, 1, 0] is just the target vector, not a gradient.

Q9 BACKPROPAGATION

A 10-layer network uses **sigmoid** activations everywhere. Ignoring the weights, what is the **upper bound** on the product of the activation derivatives that scale the gradient reaching layer 1?

- **A.** About 2.5 (10 × 0.25)
- **B.** About 0.025
- ✓ **C. About 10^{-6} ($0.25^{10} \approx 9.5\times10^{-7}$)**
- **D.** Exactly 1

Correct answer: C

Why it is correct. The sigmoid derivative $\sigma'(z) = \sigma(1 - \sigma)$ never exceeds **0.25**. Through 10 layers the gradient can be scaled by at most $0.25^{10} \approx 9.5\times10^{-7}$, which is the **vanishing-gradient** problem and explains why ReLU replaced sigmoid in hidden layers.

Why the others fail.
✗ **A.** Derivatives multiply across layers (chain rule); they are not added.
✗ **B.** 0.025 would be 0.25 × 0.1, not 0.25 raised to the 10th power.
✗ **D.** A factor of 1 is the *maximum for tanh* (at 0) or ReLU (for $z > 0$), not for sigmoid.

Q10 BACKPROPAGATION

While training an RNN the loss suddenly becomes **NaN** and the gradient norm spiked to a huge value just before. What is the most direct mitigation?

- **A.** Replace tanh by sigmoid in the recurrent layer
- ✓ **B. Gradient clipping (rescale gradients whose norm exceeds a threshold)**
- **C.** Remove the bias terms
- **D.** Increase the learning rate

Correct answer: B

Why it is correct. Exploding gradients make a single update enormous and push weights into overflow. **Clipping the gradient norm** (e.g. to 1.0–5.0) bounds the step size while keeping the direction. Lowering the learning rate and careful initialization also help.

Why the others fail.
✗ **A.** Sigmoid has smaller gradients (≤ 0.25) but is not centred, and it would slow learning rather than address the spike directly.
✗ **C.** Biases are not the source of the instability.
✗ **D.** A larger learning rate would make the exploding updates worse.

C Activation Functions

Q11 ACTIVATIONS

For the ReLU activation, what are the derivatives at $z = -2$ and $z = 3$, respectively?

- ✓ **A. 0 and 1**
- **B.** −2 and 3
- **C.** 1 and 1
- **D.** 0 and 3

Correct answer: A

Why it is correct. ReLU(z) = max(0, z). For $z < 0$ it is flat, so the derivative is **0**; for $z > 0$ it is the line $y = z$, so the derivative is **1**. This constant gradient for positive inputs is why ReLU avoids saturation.

Why the others fail.

✗ **B.** −2 and 3 are the inputs themselves, not the slopes.
✗ **C.** The slope for negative inputs is 0, not 1; that is what produces “dead” neurons.
✗ **D.** The slope for positive inputs is a constant 1, not equal to z.

Q12 ACTIVATIONS

What are $\sigma(0)$ and $\sigma'(0)$ for the sigmoid function?

- **A.** 0.5 and 0.5
- **B.** 0 and 1
- **C.** 1 and 0.25
- ✓ **D. 0.5 and 0.25**

Correct answer: D

Why it is correct. $\sigma(0) = 1/(1 + e^{0}) = 0.5$. The derivative is $\sigma(1 - \sigma) = 0.5 \cdot 0.5 = \mathbf{0.25}$, which is the maximum of the sigmoid’s slope; away from 0 it only gets smaller.

Why the others fail.

✗ **A.** $\sigma'(0) = \sigma(1 - \sigma) = 0.25$, not 0.5.
✗ **B.** 0 and 1 are the limits of σ and tanh′ at certain inputs, not the values at $z = 0$.
✗ **C.** $\sigma(0) = 0.5$; the output reaches 1 only in the limit $z \to \infty$.

Q13 ACTIVATIONS

After a large learning-rate update, about **40% of the hidden ReLU units output exactly 0 for every training example** and never recover. What is this called, and what helps?

- **A.** Overfitting; add more ReLU layers
- ✓ **B. Dying ReLU; use Leaky ReLU/GELU, a lower learning rate or He initialization**
- **C.** Saturation; switch to sigmoid activations
- **D.** Exploding gradients; remove normalization

Correct answer: B

Why it is correct. If a unit's pre-activation becomes negative for all inputs its gradient is 0, so its weights stop updating: a **dead ReLU**. Leaky ReLU keeps a small slope for $z < 0$, while a smaller learning rate and He initialization reduce the chance of pushing units into the dead zone.

Why the others fail.
✗ **A.** Overfitting is a train–validation gap and is unrelated to units being permanently inactive.
✗ **C.** Sigmoid saturates more severely and would worsen gradient flow.
✗ **D.** Dead units have zero gradient, the opposite of exploding gradients; removing normalization does not revive them.

Q14 ACTIVATIONS

An image-tagging model must predict independently whether each photo contains **cat**, **dog** or **car** (any combination is possible). Which output layer is appropriate?

- **A.** Three output units with one softmax
- **B.** A single ReLU unit
- ✓ **C. Three output units, each with its own sigmoid**
- **D.** Three tanh units whose outputs are summed

Correct answer: C

Why it is correct. This is **multi-label** classification: labels are not mutually exclusive, so each needs its own probability in [0, 1]. Use a sigmoid per unit with binary cross-entropy per label.

Why the others fail.
✗ **A.** Softmax forces the probabilities to sum to 1, implying exactly one label is present.
✗ **B.** One ReLU unit outputs a single unbounded number and cannot represent three labels.
✗ **D.** Summing outputs discards which label fired; tanh also gives values in $(-1, 1)$, not probabilities.

Q15 ACTIVATIONS

Adding the same constant (for example 100) to **every** logit before applying softmax changes the output probabilities how, and why does this matter in implementations?

- ✓ **A. It does not change them (shift invariance); implementations subtract max(z) to avoid overflow**
- ○ **B.** It makes all probabilities equal
- ○ **C.** It converts them to one-hot vectors
- ○ **D.** It doubles the probability of the largest logit

Correct answer: A

Why it is correct. $\text{softmax}(z + c)_i = e^{z_i+c} / \Sigma e^{z_j+c} = e^{z_i} / \Sigma e^{z_j}$ because e^c cancels. Therefore frameworks subtract max(z) first, so the largest exponent is $e^0 = 1$ and **overflow is avoided** without altering the result.

Why the others fail.

✗ **B.** The relative differences between logits are unchanged, so the probabilities are not equalized.
✗ **C.** One-hot behaviour needs scaling the logits (a temperature), not shifting them.
✗ **D.** The probabilities stay exactly the same; nothing is doubled.

D Convolutional Networks

Q16 CNN

A 28×28 grayscale image passes through a convolution with a **5×5** kernel, **stride 1** and **padding 2**. What is the output height and width?

- ○ **A.** 24
- ○ **B.** 26
- ○ **C.** 32
- ✓ **D. 28**

Correct answer: D

Why it is correct. $O = \lfloor (W - K + 2P)/S \rfloor + 1 = (28 - 5 + 4)/1 + 1 = \mathbf{28}$. With a K×K kernel (K odd), stride 1 and $P = (K - 1)/2$ the spatial size is preserved (“same” padding).

Why the others fail.

✗ **A.** 24 is the result with no padding: $(28 - 5)/1 + 1$.
✗ **B.** 26 would arise from a 3×3 kernel without padding or from miscounting the padding term.
✗ **C.** 32 would require padding to *add* more than the kernel removes ($P = 4$).

Q17 CNN

A convolutional layer has **128 filters** of size **3×3** applied to an input with **64 channels**, with one bias per filter. How many parameters does it have?

- ✓ **A. 73,856**
- **B.** 73,728
- **C.** 1,152
- **D.** 147,712

Correct answer: A

Why it is correct. Each filter has 3×3×64 = 576 weights plus 1 bias = 577 parameters. With 128 filters: 577 × 128 = **73,856**. The count does *not* depend on the image height or width because the weights are shared across positions.

Why the others fail.
✗ **B.** 73,728 = 576 × 128 leaves out the 128 biases.
✗ **C.** 1,152 = 3·3·128 ignores the number of input channels (64).
✗ **D.** 147,712 is exactly double; it wrongly counts every weight twice.

Q18 CNN

A feature map of size **12×12×32** goes through **2×2 max pooling with stride 2**. What is the output shape and the number of trainable parameters in the pooling layer?

- **A.** 6×6×32 and 64 parameters
- **B.** 12×12×16 and 0 parameters
- ✓ **C. 6×6×32 and 0 parameters**
- **D.** 6×6×64 and 0 parameters

Correct answer: C

Why it is correct. Pooling halves height and width (12/2 = 6) and works on each channel independently, so channels stay at 32: **6×6×32**. Max pooling just selects the maximum in each window, so it has **no trainable parameters**.

Why the others fail.
✗ **A.** Pooling has nothing to learn; the 64 is a made-up count.
✗ **B.** Pooling does not reduce the number of channels; it only reduces spatial size.
✗ **D.** Channels never double in a pooling layer; only a convolution with more filters can change the depth.

Q19 CNN

Why does a CNN need **far fewer parameters** than a fully-connected network on the same image?

- **A.** Because ReLU removes many weights
- ✓ **B. Weight sharing (the same small filter is reused at every position) combined with local connectivity**
- **C.** Because batch normalization prunes the network
- **D.** Because pooling layers contain few weights

Correct answer: B

Why it is correct. A small kernel connects each output only to a local patch (local connectivity), and the **same** kernel weights are applied at every location (weight sharing). A 3×3 filter has 9 weights regardless of image size, whereas a dense layer needs one weight per input–output pair.

Why the others fail.
✗ **A.** ReLU is an activation; it does not delete parameters.
✗ **C.** Batch norm adds a few scale/shift parameters and does not prune weights.
✗ **D.** Pooling has zero parameters, but that is a small effect compared with weight sharing in the convolution layers.

Q20 CNN

A **residual (skip) connection** computes y = F(x) + x. What is its main benefit for very deep networks?

- **A.** It halves the number of parameters
- **B.** It removes the need for activation functions
- **C.** It replaces the convolution operation
- ✓ **D. The identity path lets gradients flow directly to earlier layers, making very deep networks trainable**

Correct answer: D

Why it is correct. Since $\partial y/\partial x = \partial F/\partial x + \mathbf{1}$, the gradient always has a direct route back through the identity term, which counteracts vanishing gradients and the “degradation” problem. This enabled ResNets with 50–150+ layers.

Why the others fail.
✗ **A.** The block still contains the same layers F, so parameter count is not reduced.
✗ **B.** Residual blocks still use non-linear activations inside F.
✗ **C.** F usually *consists of* convolutions; the skip connection is added on top, not instead.

E Recurrent Networks

Q21 RNN

Why does a **vanilla RNN** have difficulty learning dependencies across many time steps?

- **A.** It cannot process variable-length sequences
- ✓ **B. Backpropagation through time multiplies the gradient by W_h and $\tanh'$ at every step, so it vanishes (or explodes) exponentially**
- **C.** It uses too many parameters
- **D.** It has no hidden state

Correct answer: B

Why it is correct. The gradient reaching step $t-k$ contains a product of k Jacobians ($W_h^\top \operatorname{diag}(\tanh')$). If their typical magnitude is below 1 the signal decays like γ^k; above 1 it explodes. Gated cells (LSTM/GRU) and clipping were introduced to address this.

Why the others fail.

✗ **A.** RNNs handle variable-length input naturally by repeating the same cell.
✗ **C.** Weight sharing across time keeps an RNN small; parameter count is not the problem.
✗ **D.** The hidden state is the core of an RNN; its repeated use causes the gradient issue.

Q22 RNN

In an LSTM, at some time step the **forget gate is ≈ 1** and the **input gate is ≈ 0**. What happens to the cell state c_t?

- ✓ **A. $c_t \approx c_{t-1}$: the memory is preserved unchanged**
- **B.** c_t is reset to 0
- **C.** c_t is replaced by the new candidate
- **D.** c_t is multiplied by the output gate

Correct answer: A

Why it is correct. The update is $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$. With $f \approx 1$ and $i \approx 0$ this becomes $c_t \approx c_{t-1}$. This **additive, gated** path lets information and gradients travel across many steps.

Why the others fail.

✗ **B.** Resetting would need $f \approx 0$ (forget everything).
✗ **C.** Replacing memory with the candidate needs $f \approx 0$ and $i \approx 1$.
✗ **D.** The output gate controls what is exposed as $h_t = o_t \odot \tanh(c_t)$; it does not change c_t itself.

Q23 RNN

An LSTM layer takes an input of size **d = 10** and has **h = 20** hidden units (with biases). How many parameters does it have?

- **A.** 620
- **B.** 1,860
- **C.** 2,400
- ✓ **D. 2,480**

Correct answer: D

Why it is correct. An LSTM has four weight sets (forget, input, output, candidate). Each has h·(h + d) weights + h biases = 20·30 + 20 = 620. Total = 4 × 620 = **2,480**.

Why the others fail.

✗ **A.** 620 is only one gate's parameters.
✗ **B.** 1,860 = 3 × 620 is the count for a GRU, which has three weight sets.
✗ **C.** 2,400 = 4·20·30 ignores the 4·20 = 80 bias parameters.

Q24 RNN

Which RNN configuration fits **sentiment classification of a whole movie review** (a sequence of words in, one label out)?

- **A.** One-to-many: generate a sequence from a single vector
- **B.** Many-to-many aligned: one output at every word
- ✓ **C. Many-to-one: classify from the final hidden state (or a pooled state)**
- **D.** One-to-one: a feed-forward network on a single word

Correct answer: C

Why it is correct. The input is a sequence but there is a **single** prediction for the whole text, so the hidden state after the last token (or a pooled summary) feeds a classifier.

Why the others fail.

✗ **A.** One-to-many is for generation, e.g. captioning an image.
✗ **B.** Aligned many-to-many (e.g. POS tagging) outputs a label per token; here there is one label per review.
✗ **D.** A single-word network ignores context, word order and negations such as "not good".

Q25 RNN

For which task is a **bidirectional** RNN **not** suitable?

- **A.** Named-entity recognition on complete sentences
- ✓ **B. Real-time next-word prediction, where future words are not yet available**
- **C.** Offline part-of-speech tagging
- **D.** Classifying an already-complete review

Correct answer: B

Why it is correct. A bidirectional RNN needs the **entire** sequence because the backward pass starts from the end. In autoregressive or streaming prediction the future tokens do not exist yet, so only unidirectional (causal) models can be used.

Why the others fail.

✗ **A.** With the whole sentence available, looking at both left and right context helps NER.
✗ **C.** Offline tagging has access to the complete input, which is the ideal setting.
✗ **D.** A finished review is fully available, so both directions can be used.

F Loss Functions

Q26 LOSS FUNCTIONS

A 3-class softmax model outputs **[0.659, 0.242, 0.099]**, and the true class is the **first**. What is the categorical cross-entropy loss?

- ✓ **A. ≈ 0.417**
- **B.** ≈ 0.659
- **C.** ≈ 0.341
- **D.** ≈ 2.31

Correct answer: A

Why it is correct. With a one-hot target the loss is $-\ln$(probability of the true class) $= -\ln(0.659) \approx$ **0.417**. The loss falls toward 0 as that probability approaches 1.

Why the others fail.

✗ **B.** 0.659 is the probability itself; cross-entropy takes its negative logarithm.
✗ **C.** 0.341 = 1 − 0.659 is a different quantity (the probability mass on other classes).
✗ **D.** 2.31 = $-\ln(0.099)$ would be the loss if the *third* class (0.099) were the true one.

Q27 LOSS FUNCTIONS

A regression dataset contains occasional **extreme outliers**. You want a loss that is smooth near zero error but does not let outliers dominate. Which is the best choice?

- **A.** Mean squared error
- **B.** Binary cross-entropy
- **C. Huber (smooth L1) loss**
- **D.** Hinge loss

Correct answer: C

Why it is correct. Huber loss is **quadratic for small errors** (smooth, stable gradients near the optimum) and **linear for large errors**, so outliers contribute a bounded gradient. MAE is robust too but has a constant gradient that is awkward near the optimum.

Why the others fail.

✗ **A.** MSE squares the error, so a single large outlier can dominate the gradient.
✗ **B.** BCE is for probabilities and binary labels, not real-valued targets.
✗ **D.** Hinge loss is used for margin-based classification, not regression.

Q28 LOSS FUNCTIONS

Which pairing is correct for a **10-class, single-label** image classifier?

- **A.** 10 outputs with sigmoid and mean squared error
- **B.** 1 output with ReLU and binary cross-entropy
- **C.** 10 linear outputs with hinge loss on the raw one-hot vector
- **D. 10 outputs with softmax and categorical cross-entropy**

Correct answer: D

Why it is correct. Exactly one class is true, so a **softmax** output (probabilities summing to 1) with **categorical cross-entropy** is the natural, well-behaved combination; its gradient with respect to the logits is simply $\hat{y} - y$.

Why the others fail.

✗ **A.** MSE on sigmoid outputs gives weak gradients when the model is confidently wrong, and treats classes as independent.
✗ **B.** One ReLU output cannot represent ten classes, and BCE needs probabilities.
✗ **C.** Hinge on raw one-hot values is not a standard multi-class setup and gives no probabilities.

Q29 LOSS FUNCTIONS

For a binary classifier, the true label is **y = 1** but the model predicts **p = 0.1**. What is the binary cross-entropy of this sample?

- **A.** ≈ 0.105
- ✓ **B. ≈ 2.303**
- **C.** 0.1
- **D.** 0.9

Correct answer: B

Why it is correct. BCE = −[y ln p + (1 − y) ln(1 − p)] = −ln(0.1) ≈ **2.303**. Compare p = 0.9, which gives only 0.105: confident wrong predictions are punished heavily.

Why the others fail.

✗ **A.** 0.105 = −ln(0.9) is the loss when the model is *right* with p = 0.9.
✗ **C.** 0.1 is the probability, not the loss.
✗ **D.** 0.9 would be the absolute error 1 − p, which is the MAE-style quantity, not cross-entropy.

Q30 LOSS FUNCTIONS

A PyTorch-style **CrossEntropyLoss** expects raw **logits** and applies log-softmax internally. A model that already ends with a softmax layer is passed to it. What is the consequence?

- ✓ **A. Softmax is applied twice, giving squashed probabilities, weaker gradients and slower, poorer training**
- **B.** Training works faster and more stably
- **C.** The loss becomes negative
- **D.** Nothing; softmax is idempotent

Correct answer: A

Why it is correct. Applying softmax to values that are already probabilities (in [0, 1]) compresses them toward equal values, so the loss landscape flattens and gradients shrink. The fix is to output **logits** and let the loss handle softmax (which is also numerically more stable).

Why the others fail.

✗ **B.** The double application hurts rather than helps.
✗ **C.** Cross-entropy of valid probabilities is never negative.
✗ **D.** Softmax is *not* idempotent: softmax(softmax(z)) ≠ softmax(z) in general.

✓ Answer Key & Score Sheet

Q	Ans	Topic	Q	Ans	Topic
1	**C**	MLP	16	**D**	CNN
2	**A**	MLP	17	**A**	CNN
3	**D**	MLP	18	**C**	CNN
4	**B**	MLP	19	**B**	CNN
5	**D**	MLP	20	**D**	CNN
6	**B**	Backpropagation	21	**B**	RNN
7	**C**	Backpropagation	22	**A**	RNN
8	**A**	Backpropagation	23	**D**	RNN
9	**C**	Backpropagation	24	**C**	RNN
10	**B**	Backpropagation	25	**B**	RNN
11	**A**	Activations	26	**A**	Loss Functions
12	**D**	Activations	27	**C**	Loss Functions
13	**B**	Activations	28	**D**	Loss Functions
14	**C**	Activations	29	**B**	Loss Functions
15	**A**	Activations	30	**A**	Loss Functions

Score interpretation

Score	Level	Recommended action
26–30	Mastery	Move on to Module 02; try explaining each answer aloud.
20–25	Proficient	Review the sections behind your misses, then redo those questions.
13–19	Developing	Re-read the Theory Study Guide sections, work the numeric examples by hand, retake after 48 hours.
0–12	Foundation	Restart with Section 1 and 2 of the guide, then attempt the quiz again.

KEY IDEA
Track your misses by topic. If you missed 2+ questions in one part (A–F), that part is your priority for revision. Numeric questions (Q2, Q8, Q9, Q16, Q17, Q23, Q26, Q29) can all be solved with the formula sheet in Section 7 of the Module 02 Study Guide.